ENSO Dashboard — Web Deployment Plan (Demo)

Status: decisions made (§0); repo prep files added and a production build verified locally; ***not yet deployed*** *— remaining steps are the guided Cloudflare walkthrough (§5).*

0. Decisions (from the planning interview)

Question	Decision
Repo access	**Fork / push to your own GitHub account**, then connect Cloudflare to that repo
Lifespan / audience	**Long-lived, public** → scheduled monthly rebuild (§6) is in scope
Data policy	**Fresh NOAA data on each build** (committed files are the fallback)
Division of work	Claude prepares the repo; the Cloudflare/GitHub account steps are done together, step by step

Repo prep already done (uncommitted, awaiting your review): `.node-version` (`24`), `public/_headers` (cache rules), `.github/workflows/monthly-rebuild.yml` (monthly deploy-hook trigger; needs the secret from §6). `npx tsc --noEmit` and `npx vite build` pass locally; `dist/` contains `index.html`, `assets/`, `data/`, `_headers`. Not yet done: committing, creating the fork, anything in Cloudflare.

Note: the working tree also has uncommitted changes to `data-prep/raw/*` and `public/data/*` that this prep did not make (a data refresh run elsewhere); decide whether to commit that fresh snapshot with the prep files.

1. Goal and constraints

Put the dashboard on a public URL for demonstration, at **zero cost**, with minimal ongoing effort.

What the app needs from a host (from the code review — see `codebase-summary.md`):

- Serve static files only: `dist/` (HTML, JS, CSS) plus `/data/*.json`. **No server, no env vars, no secrets, no database.**
- Build command `npm run build` (runs a NOAA refresh, `tsc`, then `vite build`); output dir `dist`; Node ≥ 20 (developed on Node 24).

- JSON is fetched from absolute paths (`/data/enso-monthly.json`), so serve from a **domain root** (any `*.pages.dev`, `*.netlify.app` or custom domain works; a `/repo-name/` sub-path, e.g. default GitHub Pages project URL, would break data loading unless `vite.config` `base` and `data.ts` are changed).
- No client-side routing → no SPA-fallback rules needed.

2. Options compared

	Cloudflare Pages / Workers static assets	**Netlify**	**Vercel**	**GitHub Pages**
Free tier fit	Unlimited bandwidth & requests for static assets; 500 builds/month (Pages)	100 GB bandwidth, 300 build-minutes/month (credit-based on newer plans)	Generous, but Hobby is non-commercial only	Free, 1 GB site / 100 GB/month soft limits
Git auto-deploy	Yes	Yes	Yes	Via GitHub Actions
Preview URLs per branch/PR	Yes	Yes	Yes	No (manual)
Scheduled data refresh	Deploy hook + any cron (GitHub Actions)	Build hook + cron	Deploy hook + cron	Actions cron natively
Already have account	**Yes**	No	No	GitHub yes
Root-domain URL	Yes (`<name>.pages.dev`)	Yes	Yes	Only for `user.github.io` repos

Free-tier numbers are from memory and change; confirm on the vendor pages before relying on them. Cloudflare has been steering new projects toward "Workers with static assets" rather than classic Pages — both serve this app identically; the steps below use the Git-integrated flow, which is the same in either dashboard.

3. Recommendation

Cloudflare Pages (Git-integrated). You already have the account, bandwidth isn't a concern for a static demo, and previews per branch are useful for showing work in progress. Netlify is the equally easy fallback and needs no changes to this plan beyond the settings screen.

4. Pre-deployment checklist (repo changes)

Small, all optional except where marked.

1. **Pin the Node version** (*recommended*): add `.node-version` containing `24` (Cloudflare/Netlify both read it), or set `NODE_VERSION=24` as a build env var.
2. **Verify a clean production build locally**: `npm ci && npm run build && npm run preview`; click through every section (map, event row zoom, rate toggle, legend isolate, frequency window buttons, theme toggle) in the built bundle, not just dev.
3. **Decide on build-time NOAA refresh** (*see Q4*): default behaviour re-downloads on every CI build (stamp file is gitignored), which makes each deploy self-updating but means a build depends on NOAA being reachable. It falls back to the committed raw files if not, so it cannot hard-fail. Alternative: deploy with `npm run data:build && tsc && vite build` as the build command to pin the committed snapshot (fully reproducible demo).
4. **Housekeeping, optional**: remove the unused `rg` dependency; fix the stale doc path comment in `classify.ts`.
5. **Repo access** (*see Q1*): the `origin` remote is `github.com/rohans0/ENSO_dashboard`. Cloudflare's Git integration needs permission on whichever repo it builds from.
6. **Caching headers, optional**: add `public/_headers` so HTML revalidates but hashed assets are immutable, and the JSON refreshes within an hour:

```
/data/*
  Cache-Control: public, max-age=3600
/assets/*
  Cache-Control: public, max-age=31536000, immutable
```

5. Deployment steps — Cloudflare Pages

1. Make the code available to Cloudflare: either (a) authorize the Cloudflare GitHub app on the repo `rohans0/ENSO_dashboard`, or (b) fork/push to your own GitHub account and authorize that (see Q1).
2. Cloudflare dashboard → **Workers & Pages** → **Create** → **Pages** → **Connect to Git** → select the repo, branch `main`.
3. Build settings:
   - Framework preset: **None** (or Vite — same values)
   - Build command: `npm run build`
   - Build output directory: `dist`
   - Root directory: *(blank)*
   - Environment variable: `NODE_VERSION = 24` (if no `.node-version` file)

4. **Save and Deploy.** First build takes ~1–2 min (installs, NOAA download, type-check, bundle). The site goes live at `https://<project>.pages.dev`.
5. Smoke test the live URL (checklist in §7).
6. *(Optional)* Custom domain: Pages project → Custom domains → add a domain/subdomain on your Cloudflare zone; DNS and TLS are automatic.

Alternative without Git integration (no repo permissions needed): `npm run build`, then upload with `npx wrangler pages deploy dist --project-name enso-dashboard` (requires `wrangler login`, run from your machine). Fastest for a one-off demo; no automatic updates.

6. Keeping data fresh (optional, demo-grade)

NOAA indices change monthly. Two ways to keep the demo current without touching it:

- **Scheduled rebuild (recommended if demo will live > a month):** create a Cloudflare **Deploy Hook** (Pages project → Settings → Builds & deployments → Deploy hooks), then add a GitHub Actions workflow with `on: schedule: cron: '0 6 5 * *'` (the 5th of each month) that `curl -X POST`s the hook URL (stored as a repo secret). Each rebuild re-downloads NOAA data via `prebuild`.
- **Manual:** `npm run data:refresh:force`, commit `data-prep/raw/*` and `public/data/*`, push; Pages redeploys.

7. Verification checklist (after deploy)

- [ ] Page loads; no console errors; both `/data/enso-monthly.json` and `/data/enso-events.json` return 200 with JSON.
- [ ] Regional map draws coastlines (checks the dynamically imported `world-atlas` chunk).
- [ ] KaTeX math renders in the info panels (fonts load).
- [ ] Event-row click zooms; range slider drags; smoothing slider works.
- [ ] Rate-of-change toggle, legend isolate, frequency window buttons all work.
- [ ] Dark/light toggle persists on reload.
- [ ] Mobile width: no unusable overflow (the app has no responsive breakpoints — a known gap, acceptable for a demo but worth knowing before presenting on a phone).
- [ ] Latest ONI value in the stat tile matches NOAA's current table.

8. Risks and mitigations

Risk	Impact	Mitigation
NOAA unreachable during build	stale data	Built-in fallback to committed raw files; or pin snapshot (step 4.3)
NOAA changes a file format	parser throws → **build fails** (the fallback only covers download errors, not parse errors)	Previous successful deployment stays live; fix parser; consider pinning snapshot for demos
Wrong Node version	build error from Vite 8 / TS 6	`.node-version` / `NODE_VERSION`
Repo owned by another GitHub account	can't connect Git integration	Fork, or use direct `wrangler` upload
Free-tier limits change	n/a for static demo	Re-check vendor pages; Netlify is a drop-in alternative
Public URL exposes work-in-progress	reputational	Use a non-obvious `*.pages.dev` name; restrict previews with Cloudflare Access if needed

9. Rollback

Pages keeps every deployment: dashboard → Deployments → previous build → **Rollback to this deployment**. For Git-driven changes, `git revert` and push.

10. Estimated effort

~30–45 minutes for first deploy (mostly repo-permission setup and one build); ~15 minutes extra for the scheduled refresh.

11. Remaining questions

1. **Cloudflare setup:** classic *Pages* or the newer *Workers static assets* flow? (Whichever the dashboard offers you; settings are identical — we'll follow what you see.)
2. **Custom domain:** is `<name>.pages.dev` fine for a long-lived link, or do you want a domain on your Cloudflare zone?
3. **Pre-deploy polish:** deploy as-is, or address the layout/mobile gap (plan.md workstream 4) first?
4. **Fork name:** keep `ENSO_dashboard`, or rename? (Becomes the default project/URL name.)

12. Guided walkthrough (do together, in order)

1. **Review & commit prep files** (Claude) — decide on the data snapshot noted in §0.
2. **Create your fork** — on github.com/rohans0/ENSO_dashboard click *Fork*; or create an empty repo and `git remote add mine <url>` + `git push mine main`. *(You click, Claude supplies the commands.)*
3. **Cloudflare: connect Git** — §5 steps 2–3 (screens and exact values read out one at a time).
4. **First deploy & smoke test** — §7 checklist.
5. **Deploy hook + GitHub secret** — create hook in Cloudflare, add `CF_PAGES_DEPLOY_HOOK` under the fork's Settings → Secrets → Actions, run the workflow manually via *Run workflow* to confirm it triggers a build.
6. *(Optional)* custom domain.